

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF SCIENCE

Faculty of Information Technology



REPORT
FULLY DISTRIBUTED HADOOP CLUSTER
COURSE: INTRODUCTION TO BIG DATA
CLASS: 23KHDL2

STUDENT TEAM:

- Nguyễn Hoàng Minh Tâm – 23127017
- Phan Nhật Thiện Nhân – 23127096
- Phan Trung Nhật – 23127240
- Bùi Nam Việt – 23127516

INSTRUCTORS:

- Nguyễn Ngọc Thảo
- Huỳnh Lâm Hải Đăng
- Trần Huy Bân

Ho Chi Minh City, July 12, 2026.

Contents

1	Deployment of a Fully Distributed Hadoop Cluster on Docker Virtual Machines	3
1.1	System Architecture	3
1.2	Deployment Procedure	3
1.2.1	Step 1: Creating the Internal Docker Network	3
1.2.2	Step 2: Building the Base Docker Image	3
1.2.3	Step 3: Launching the Hadoop Virtual Machines	3
2	Detailed System Configuration and Distributed Synchronization	4
2.1	Step 1: Starting the SSH Service on the Master Node	4
2.2	Step 2: Configuring the Hadoop System Files	4
2.3	Step 3: Starting the SSH Service on the Worker Nodes	6
2.4	Step 4: Synchronizing Hadoop Configuration Across the Cluster	6
3	Formatting the NameNode and Cluster Validation	7
3.1	Formatting the NameNode	7
4	Starting and Validating the Distributed Hadoop Cluster	7
4.1	Step 1: Starting Hadoop and YARN Services	7
4.2	Step 2: Verifying Running Daemons Using JPS	7
5	Cluster Validation Through the Hadoop Web UI	8
6	Cluster Validation and HDFS Security Verification	9
6.1	Verification of Hadoop Daemons on Worker Nodes	9
6.2	HDFS Permission and Ownership Verification	9
6.3	Cluster Validation Using <code>hadoop-test.jar</code>	10

1 Deployment of a Fully Distributed Hadoop Cluster on Docker Virtual Machines

1.1 System Architecture

The deployment uses a fully distributed architecture with three independent Hadoop nodes connected through an isolated Docker network.

- **One master node:** Hosts the HDFS NameNode and the YARN ResourceManager.
- **Two worker nodes:** Each hosts an HDFS DataNode and a YARN NodeManager.

1.2 Deployment Procedure

1.2.1 Step 1: Creating the Internal Docker Network

An isolated Docker network named `hadoop-net` was created with the subnet `172.20.0.0/16`. This network allows the containers to communicate through assigned static IP addresses. The network was created as follows:

```
docker network create --subnet=172.20.0.0/16 hadoop-net
```

Figure 1 shows the network creation command and the returned network identifier.

```
PS C:\Users\ASUS> docker network create --subnet=172.20.0.0/16 hadoop-net
3fd46359317e2da4387b91f9c0784b2482cef67fd374f2811d17abca0eb99d68
```

Figure 1: Creation of the `hadoop-net` Docker network with the configured subnet.

1.2.2 Step 2: Building the Base Docker Image

A Dockerfile was prepared to automate installation of the software required by the cluster. The report specifies an Ubuntu 22.04 base image with OpenJDK, OpenSSH Server, passwordless SSH key generation, and Apache Hadoop; these contents should be verified against the Dockerfile. The image was built with the following command, and the build output is shown in Figure 2:

```
docker build -t hadoop-base .
```

```
PS D:\Documents\Github\Intro2BD-Lab1\hadoop-cluster> docker build -t hadoop-base .
[+] Building 57.3s (4/10)
-> [internal] load build definition from Dockerfile
-> => transferring dockerfile: 1.28kB
-> [internal] load metadata for docker.io/library/ubuntu:22.04
-> [internal] load .dockerignore
-> => transferring context: 2B
-> CACHED [1/7] FROM docker.io/library/ubuntu:22.04@sha256:0e0a0fc6d18feda9db1590da249ac93e8d5abfea8f4c3c0c849ce512b5ef8982
-> => resolve docker.io/library/ubuntu:22.04@sha256:0e0a0fc6d18feda9db1590da249ac93e8d5abfea8f4c3c0c849ce512b5ef8982
[+] Building 57.8s (4/10)
-> [internal] load build definition from Dockerfile
-> => transferring dockerfile: 1.28kB
-> [internal] load metadata for docker.io/library/ubuntu:22.04
-> [internal] load .dockerignore
```

Figure 2: Build process for the `hadoop-base` Docker image.

1.2.3 Step 3: Launching the Hadoop Virtual Machines

Three Docker containers were launched from the `hadoop-base` image to represent separate Hadoop nodes. Each container received a unique hostname and static IP address on `hadoop-net`.

- **hadoop-master (172.20.0.2):** Master node, with host mappings for port 9870 (NameNode Web UI) and port 8088 (ResourceManager Web UI).
- **hadoop-worker1 (172.20.0.3):** Worker node 1.
- **hadoop-worker2 (172.20.0.4):** Worker node 2.

Figure 3 records the container startup commands and the resulting `docker ps` output.

```

C:\Users\ASUS>docker run -d --name hadoop-master --net hadoop-net --ip 172.20.0.2 -h hadoop-master -p 9870:9870 -p 8088:8088 hadoop
-base tail -f /dev/null
8d7e6a7c3a504afae10ba86a9b1aa62d313cbc67b5a3badec3ff25cfba16b170

C:\Users\ASUS>docker run -d --name hadoop-worker1 --net hadoop-net --ip 172.20.0.3 -h hadoop-worker1 hadoop-base tail -f /dev/null
b51f519937529ce43f9c89235f63fffe59fda3baf1879db0a29868b8785fafc2

C:\Users\ASUS>docker run -d --name hadoop-worker2 --net hadoop-net --ip 172.20.0.4 -h hadoop-worker2 hadoop-base tail -f /dev/null
8aaeb60261f84af2a0e3333c9946fae7f69d7c727314ad5d82a00754839519c9

C:\Users\ASUS>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
8aaeb60261f8   hadoop-base    "tail -f /dev/null"      3 seconds ago Up 3 seconds  8088/tcp, 9000/tcp, 9870/tcp
b51f51993752   hadoop-base    "tail -f /dev/null"      9 seconds ago Up 9 seconds  8088/tcp, 9000/tcp, 9870/tcp
8d7e6a7c3a50   hadoop-base    "tail -f /dev/null"      14 seconds ago Up 14 seconds  0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp, 0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp
hadoop-master

```

Figure 3: Startup and status verification of the master and worker containers.

2 Detailed System Configuration and Distributed Synchronization

2.1 Step 1: Starting the SSH Service on the Master Node

The `hadoop-master` container was accessed as `root`. The SSH process directory was created at `/var/run/sshd`, and the OpenSSH service was started. The session was then changed to the `hadoop` account. As Figure 4 also shows unsuccessful `sudo` authentication attempts after this change, any command requiring `sudo` under this account should be verified.

```

C:\Users\ASUS>docker exec -u 0 -it hadoop-master bash
root@hadoop-master:/home/hadoop# mkdir -p /var/run/sshd
root@hadoop-master:/home/hadoop# service ssh start
* Starting OpenBSD Secure Shell server sshd
root@hadoop-master:/home/hadoop# su - hadoop
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

hadoop@hadoop-master:~$ sudo su - hadoop
[sudo] password for hadoop:
Sorry, try again.
[sudo] password for hadoop:
Sorry, try again.
[sudo] password for hadoop:
sudo: 3 incorrect password attempts
hadoop@hadoop-master:~$ |

```

Figure 4: SSH service startup on the master container and switch to the `hadoop` user.

2.2 Step 2: Configuring the Hadoop System Files

The Hadoop configuration directory was accessed using:

```
cd $HADOOP_HOME/etc/hadoop/
```

The following configuration files were edited to define the distributed Hadoop cluster.
workers

The `workers` file specifies the Worker nodes participating in the cluster. The default entry `localhost` was removed and replaced with:

```
hadoop-worker1
hadoop-worker2
```

core-site.xml

The `core-site.xml` file configures the default HDFS NameNode endpoint.

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://hadoop-master:9000</value>
  </property>
</configuration>
```

hdfs-site.xml

The `hdfs-site.xml` file specifies the HDFS replication factor. Since the cluster contains two Worker nodes, the replication factor was configured as **2**.

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>2</value>
  </property>
</configuration>
```

yarn-site.xml

The `yarn-site.xml` file configures YARN to use the Master node as the centralized ResourceManager.

```
<configuration>
  <property>
    <name>yarn.resourcemanager.hostname</name>
    <value>hadoop-master</value>
  </property>

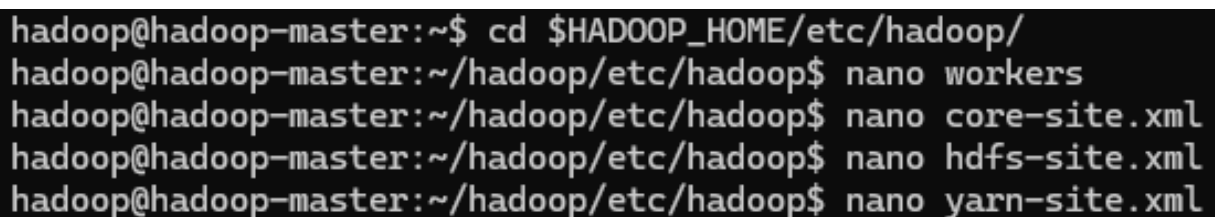
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

hadoop-env.sh

The Hadoop environment file was updated to specify the Java installation directory.

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

Figure 5 illustrates the process of editing the Hadoop configuration files.



```
hadoop@hadoop-master:~$ cd $HADOOP_HOME/etc/hadoop/
hadoop@hadoop-master:~/hadoop/etc/hadoop$ nano workers
hadoop@hadoop-master:~/hadoop/etc/hadoop$ nano core-site.xml
hadoop@hadoop-master:~/hadoop/etc/hadoop$ nano hdfs-site.xml
hadoop@hadoop-master:~/hadoop/etc/hadoop$ nano yarn-site.xml
```

Figure 5: Editing the Hadoop configuration files, including `workers`, `core-site.xml`, `hdfs-site.xml`, `yarn-site.xml`, and `hadoop-env.sh`.

2.3 Step 3: Starting the SSH Service on the Worker Nodes

On both worker containers, the SSH process directory was created and the `sshd` service was started. This enables Hadoop startup scripts on the master node to launch daemons remotely through SSH, as documented in Figure 6.

```
PS C:\Users\ASUS> docker exec -u 0 -it hadoop-worker1 bash
root@hadoop-worker1:/home/hadoop# mkdir -p /var/run/sshd && service ssh start
* Starting OpenBSD Secure Shell server sshd
root@hadoop-worker1:/home/hadoop# exit
exit
PS C:\Users\ASUS> docker exec -u 0 -it hadoop-worker2 bash
root@hadoop-worker2:/home/hadoop# mkdir -p /var/run/sshd && service ssh start
* Starting OpenBSD Secure Shell server sshd
root@hadoop-worker2:/home/hadoop# exit
exit
```

Figure 6: SSH service startup on both worker containers.

2.4 Step 4: Synchronizing Hadoop Configuration Across the Cluster

From `hadoop-master`, `scp` copied the Hadoop configuration directory to each worker. The first connections added the worker host keys to the master's `known_hosts` file. Figures 7 and 8 show the transfers; whether every source and destination file remained identical afterward should be verified.

```
hadoop@hadoop-master:~/hadoop/etc/hadoop$ scp -r $HADOOP_HOME/etc/hadoop hadoop-worker1:/home/hadoop/hadoop/etc/
The authenticity of host 'hadoop-worker1 (172.28.0.3)' can't be established.
ED25519 key fingerprint is SHA256:I3EFQ691hsk6LnZFw5LpZ906H2UcJgvnTeU5shGf8hM.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'hadoop-worker1' (ED25519) to the list of known hosts.
hadoop-worker1:/home/hadoop/hadoop/etc/
mapred-env.cmd 100% 951 3.3MB/s 00:00
hadoop-metrics2.properties 100% 3321 13.9MB/s 00:00
ssl-client.xml.example 100% 2316 9.9MB/s 00:00
hadoop-env.sh 100% 16KB 30.1MB/s 00:00
https-log4j.properties 100% 1657 7.2MB/s 00:00
user_ec_policies.xml.template 100% 2681 9.6MB/s 00:00
ssl-server.xml.example 100% 3766 14.0MB/s 00:00
mapred-site.xml 100% 758 2.7MB/s 00:00
mapred-queues.xml.template 100% 4113 15.9MB/s 00:00
kms-env.sh 100% 1351 6.2MB/s 00:00
https-env.sh 100% 1484 4.0MB/s 00:00
core-site.xml 100% 888 3.0MB/s 00:00
log4j.properties 100% 14KB 42.8MB/s 00:00
hadoop-policy.xml 100% 14KB 45.7MB/s 00:00
kms-site.xml 100% 682 3.2MB/s 00:00
hdfs-rbf-site.xml 100% 683 2.7MB/s 00:00
hadoop-user-functions.sh.example 100% 3414 10.3MB/s 00:00
yarn-env.sh 100% 7095 26.4MB/s 00:00
mapred-env.sh 100% 1764 6.8MB/s 00:00
```

Figure 7: SCP synchronization of the Hadoop configuration to `hadoop-worker1`.

```
hadoop@hadoop-master:~/hadoop/etc/hadoop$ scp -r $HADOOP_HOME/etc/hadoop hadoop-worker2:/home/hadoop/hadoop/etc/
The authenticity of host 'hadoop-worker2 (172.28.0.4)' can't be established.
ED25519 key fingerprint is SHA256:I3EFQ691hsk6LnZFw5LpZ906H2UcJgvnTeU5shGf8hM.
This host key is known by the following other names/addresses:
./ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'hadoop-worker2' (ED25519) to the list of known hosts.
hadoop-worker2:/home/hadoop/hadoop/etc/
mapred-env.cmd 100% 951 2.3MB/s 00:00
hadoop-metrics2.properties 100% 3321 10.3MB/s 00:00
ssl-client.xml.example 100% 2316 9.1MB/s 00:00
hadoop-env.sh 100% 16KB 26.9MB/s 00:00
https-log4j.properties 100% 1657 6.8MB/s 00:00
user_ec_policies.xml.template 100% 2681 7.8MB/s 00:00
ssl-server.xml.example 100% 3766 9.7MB/s 00:00
mapred-site.xml 100% 758 2.2MB/s 00:00
mapred-queues.xml.template 100% 4113 11.7MB/s 00:00
kms-env.sh 100% 1351 3.2MB/s 00:00
https-env.sh 100% 1484 2.9MB/s 00:00
core-site.xml 100% 888 3.6MB/s 00:00
log4j.properties 100% 14KB 49.2MB/s 00:00
hadoop-policy.xml 100% 14KB 51.4MB/s 00:00
kms-site.xml 100% 682 2.8MB/s 00:00
hdfs-rbf-site.xml 100% 683 1.3MB/s 00:00
hadoop-user-functions.sh.example 100% 3414 10.6MB/s 00:00
yarn-env.sh 100% 7095 24.8MB/s 00:00
mapred-env.sh 100% 1764 6.3MB/s 00:00
```

Figure 8: SCP synchronization of the Hadoop configuration to `hadoop-worker2`.

The environment script was subsequently copied to both workers, as shown in Figure 9.

```
hadoop@hadoop-master:~/hadoop/etc/hadoop$ nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
hadoop@hadoop-master:~/hadoop/etc/hadoop$ scp $HADOOP_HOME/etc/hadoop/hadoop-env.sh hadoop-worker1:/home/hadoop/hadoop/etc/hadoop/
hadoop@hadoop-master:~/hadoop/etc/hadoop$ scp $HADOOP_HOME/etc/hadoop/hadoop-env.sh hadoop-worker2:/home/hadoop/hadoop/etc/hadoop/
hadoop@hadoop-master:~/hadoop/etc/hadoop$
```

Figure 9: Synchronization of `hadoop-env.sh` to both worker nodes.

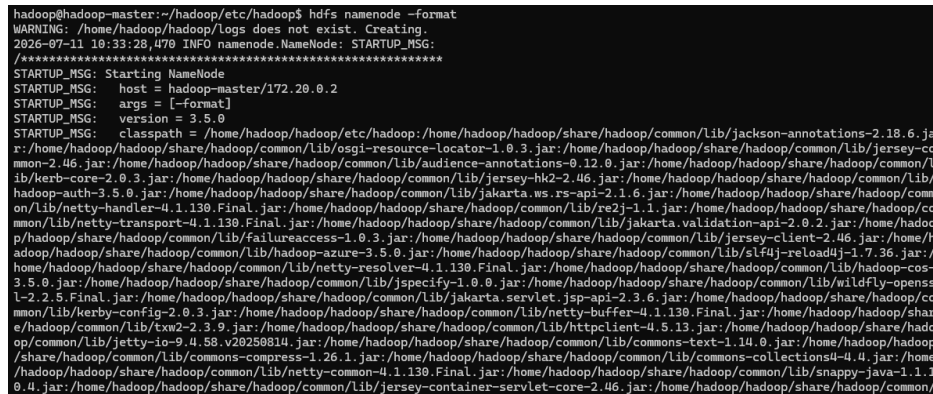
3 Formatting the NameNode and Cluster Validation

3.1 Formatting the NameNode

After configuration synchronization, the NameNode metadata storage was initialized by formatting HDFS on the master node:

```
hdfs namenode -format
```

Figure 10 shows the beginning of the formatting process. Successful completion and generation of a unique `ClusterID` should be verified from the full command output before the cluster is used.



```
hadoop@hadoop-master:~/hadoop/etc/hadoop$ hdfs namenode -format
WARNING: /home/hadoop/hadoop/logs does not exist. Creating.
2026-07-11 10:33:28.470 INFO namenode.NameNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NameNode
STARTUP_MSG: host = hadoop-master/172.29.0.2
STARTUP_MSG: args = [-format]
STARTUP_MSG: version = 3.5.0
STARTUP_MSG: classpath = /home/hadoop/hadoop/etc/hadoop:/home/hadoop/hadoop/share/hadoop/common/lib/jackson-annotations-2.18.6.jar:/home/hadoop/hadoop/share/hadoop/common/lib/osgi-resource-locator-1.0.3.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jersey-core-2.46.jar:/home/hadoop/hadoop/share/hadoop/common/lib/audience-annotations-0.12.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/kerb-core-2.0.3.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jersey-hk2-2.46.jar:/home/hadoop/hadoop/share/hadoop/common/lib/hadoop-auth-3.5.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jakarta.ws.rs-api-2.1.6.jar:/home/hadoop/hadoop/share/hadoop/common/lib/netty-handler-4.1.130.Final.jar:/home/hadoop/hadoop/share/hadoop/common/lib/re2j-1.1.jar:/home/hadoop/hadoop/share/hadoop/common/lib/netty-transport-4.1.130.Final.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jakarta.validation-api-2.0.2.jar:/home/hadoop/hadoop/share/hadoop/common/lib/failureaccess-1.0.3.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jersey-client-2.46.jar:/home/hadoop/hadoop/share/hadoop/common/lib/hadoop-azure-3.5.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar:/home/hadoop/hadoop/share/hadoop/common/lib/netty-resolver-4.1.130.Final.jar:/home/hadoop/hadoop/share/hadoop/common/lib/hadoop-cos-3.5.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jspecify-1.0.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/wildfly-openssl-2.2.5.Final.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jakarta.servlet.jsp-api-2.3.6.jar:/home/hadoop/hadoop/share/hadoop/common/lib/kerby-config-2.0.3.jar:/home/hadoop/hadoop/share/hadoop/common/lib/netty-buffer-4.1.130.Final.jar:/home/hadoop/hadoop/share/hadoop/common/lib/cxw2-2.3.9.jar:/home/hadoop/hadoop/share/hadoop/common/lib/httpclient-4.5.13.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jetty-io-9.4.58.v20250814.jar:/home/hadoop/hadoop/share/hadoop/common/lib/commons-text-1.14.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/commons-compress-1.26.1.jar:/home/hadoop/hadoop/share/hadoop/common/lib/commons-collections4-4.4.jar:/home/hadoop/hadoop/share/hadoop/common/lib/netty-common-4.1.130.Final.jar:/home/hadoop/hadoop/share/hadoop/common/lib/snappy-java-1.1.10.4.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jersey-container-servlet-core-2.46.jar:/home/hadoop/hadoop/share/hadoop/common/
```

Figure 10: Beginning of the HDFS NameNode formatting process.

4 Starting and Validating the Distributed Hadoop Cluster

4.1 Step 1: Starting Hadoop and YARN Services

The HDFS and YARN services were started with Hadoop's startup scripts:

```
start-dfs.sh
start-yarn.sh
```

The output in Figure 11 reports that missing worker log directories were created during startup. The screenshot does not show an associated failure, but daemon logs should be checked if these warnings recur.

4.2 Step 2: Verifying Running Daemons Using JPS

The Java Process Status tool (JPS), invoked with `jps`, was used on the master node to list its running Java processes. Figure 11 shows the NameNode, SecondaryNameNode, and ResourceManager processes on that node.

The master-node output contains the following Hadoop daemons:

- **NameNode** – HDFS metadata management.
- **SecondaryNameNode** – Metadata checkpoint service.
- **ResourceManager** – Centralized YARN resource scheduler.

The SecondaryNameNode periodically creates namespace checkpoints; it is not a standby NameNode. The DataNode and NodeManager processes should also be verified with JPS on each worker because they do not appear in the master-node listing.

```

hadoop@hadoop-master:~/hadoop/etc/hadoop$ start-dfs.sh
Starting namenodes on [hadoop-master]
Starting datanodes
hadoop-worker1: WARNING: /home/hadoop/hadoop/logs does not exist. Creating.
hadoop-worker2: WARNING: /home/hadoop/hadoop/logs does not exist. Creating.
Starting secondary namenodes [hadoop-master]
hadoop@hadoop-master:~/hadoop/etc/hadoop$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hadoop@hadoop-master:~/hadoop/etc/hadoop$ jps
1252 ResourceManager
1606 Jps
729 NameNode
975 SecondaryNameNode
hadoop@hadoop-master:~/hadoop/etc/hadoop$ |

```

Figure 11: Hadoop service startup and JPS daemon verification on the master node.

5 Cluster Validation Through the Hadoop Web UI

After starting all Hadoop services, the cluster status was verified through the Hadoop Web interfaces using the forwarded ports on the Windows host machine.

The HDFS NameNode Web UI was accessed at:

```
http://localhost:9870
```

As shown in Figure 12, the NameNode summary reports:

- **Configured Capacity:** Approximately **1.97 TB**.
- **Live Nodes:** Two active DataNodes and no dead nodes.
- **Safe Mode:** **OFF**, indicating that the cluster is ready to process read and write requests.

Summary

Security is off.	
Safemode is off.	
1 files and directories, 0 blocks (0 replicated blocks, 0 erasure coded block groups) = 1 total filesystem object(s).	
Heap Memory used 75.77 MB of 117 MB Heap Memory. Max Heap Memory is 1.89 GB.	
Non Heap Memory used 67.91 MB of 71.63 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.	
Configured Capacity:	1.97 TB
Configured Remote Capacity:	0 B
DFS Used:	48 KB (0%)
Non DFS Used:	35.07 GB
DFS Remaining:	1.83 TB (93.17%)
Block Pool Used:	48 KB (0%)
DataNodes usages% (Min/Median/Max/stdDev):	0.00% / 0.00% / 0.00% / 0.00%
Live Nodes	2 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	0
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Sat Jul 11 17:39:23 +0700 2026
Last Checkpoint Time	Sat Jul 11 17:33:29 +0700 2026
Last HA Transition Time	Never
Enabled Erasure Coding Policies	RS-6-3-1024k

Figure 12: HDFS NameNode Web UI showing the cluster capacity, Safe Mode status, and active DataNodes.

The YARN ResourceManager Web UI was then accessed to verify the resource management services. As illustrated in Figure 13, the ResourceManager was running normally and

detected two active Worker nodes.

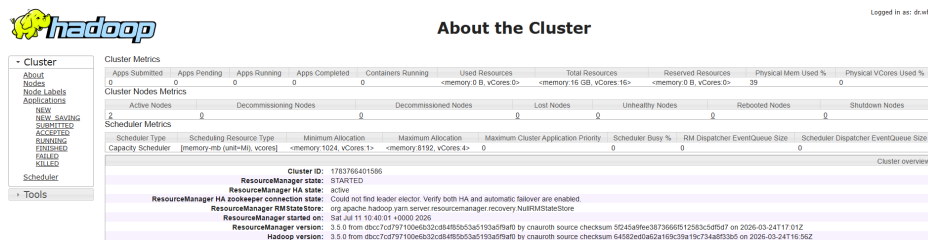


Figure 13: YARN ResourceManager Web UI displaying the cluster overview and two active Worker nodes.

6 Cluster Validation and HDFS Security Verification

6.1 Verification of Hadoop Daemons on Worker Nodes

The Hadoop services running on each Worker node were verified locally using the Java Process Status tool (jps). Both Worker nodes were found to be operating correctly.

- **hadoop-worker1:** Running the DataNode and NodeManager daemons.
- **hadoop-worker2:** Running the DataNode and NodeManager daemons.

Figure 14 confirms that the required Hadoop daemons were successfully started on both Worker nodes.

```
C:\Users\ASUS>docker exec -u hadoop -it hadoop-worker1 jps
71 DataNode
217 NodeManager
444 Jps

C:\Users\ASUS>docker exec -u hadoop -it hadoop-worker2 jps
72 DataNode
217 NodeManager
443 Jps
```

Figure 14: Verification of Hadoop daemons on both Worker nodes using the jps command.

6.2 HDFS Permission and Ownership Verification

To verify HDFS access control, a sample file was uploaded to the directory `/hcmus/23127516/`. The directory permission was then changed to 744, and the ownership was assigned to the user `khtn_23127516`.

```
hdfs dfs -chmod 744 /hcmus/23127516
hdfs dfs -chown khtn_23127516:supergroup /hcmus/23127516
hdfs dfs -ls -d /hcmus/23127516
```

The resulting directory information is shown below.

```
drwxr--r-- - khtn_23127516 supergroup 0 2026-07-12 08:41 /hcmus/23127516
```

Figure 15 confirms that the permission and ownership settings were successfully applied.

```
hadoop@hadoop-master:~$ echo "Fully distributed on virtual machines" > /home/hadoop/sample.txt
hadoop@hadoop-master:~$ hdfs dfs -put /home/hadoop/sample.txt /hcmus/23127516/
hadoop@hadoop-master:~$ hdfs dfs -chmod 744 /hcmus/23127516
hadoop@hadoop-master:~$ hdfs dfs -chown khtn_23127516 /hcmus/23127516
hadoop@hadoop-master:~$ hdfs dfs -ls /hcmus/
Found 1 items
drwxr--r--  - khtn_23127516 supergroup          0 2026-07-12 08:41 /hcmus/23127516
hadoop@hadoop-master:~$
```

Figure 15: Verification of HDFS directory permissions and ownership.

6.3 Cluster Validation Using `hadoop-test.jar`

Finally, the deployment was validated using the automated verification tool `hadoop-test.jar`.

```
java -jar /home/hadoop/hadoop-test.jar 9000 /hcmus/23127516
```

The validation tool successfully:

- Connected to the NameNode service.
- Located the uploaded file in HDFS.
- Verified the directory ownership.
- Generated the verification file `/home/hadoop/23127516_verification.txt`.

Figure 16 presents the successful execution of the automated validation tool.

```
hadoop@hadoop-master:~$ java -jar /home/hadoop/hadoop-test.jar 9000 /hcmus/23127516
Trying to read /hcmus/23127516
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Found hdfs://localhost:9000/hcmus/23127516/sample.txt
Your student ID: 23127516 (ensure it matches your student ID)
File written at /home/hadoop/23127516_verification.txt
```

Figure 16: Successful validation of the Hadoop deployment using `hadoop-test.jar`.